

Training a 30M-Parameter Small Language Model from Scratch for English Children’s Fables

A reproducible study: can a tiny model rival a large LLM on constrained story generation?

trieulh - IT5410

2026-07-16

Contents

Abstract	2
1. Introduction and thesis	3
2. Scientific grounding	3
3. Dataset	3
4. Model and training recipe	3
5. Experiments (from step 0)	4
6. Training dynamics	5
7. Language-modeling quality	5
8. Application analysis (intrinsic, reference-free)	5
9. Findings by stage	11
10. Automatic verdict (Phase 2)	12
11. Strengths and limitations	15
12. Future directions (with feasibility evidence)	15
13. Conclusion	16
14. Reproducibility	16
References	16

Abstract

We train a **30M-parameter Llama-style language model from scratch** on the TF1-EN-3M children’s-fable corpus and study, end to end and reproducibly, whether such a tiny model can approach the output quality of a 130x larger instruction-tuned LLM (Qwen3-4B) on a *constrained* generation task. Following modern scaling-law practice, we move from a deliberately under-trained baseline (loss 1.8) through a Chinchilla-aware retraining (loss 1.278, held-out perplexity 3.56) and a targeted data intervention that cures a template-collapse failure (“wise old owl” generation rate 90% -> 23%). We add an application-analysis dashboard (intrinsic diversity, readability, Zipf, positional loss) and close with **DPO preference alignment (RLAIF)** that lifts prompt-adherence while preserving language quality (perplexity drift 0%). The final 30M model generates coherent, complete fables at **~949 tokens/s, roughly 50x faster than the 4B reference**, reaching an LLM-judge quality of ~7.0/10 (from 2.5 at baseline). All parameters, curves and artifacts are reported from training step 0.

1. Introduction and thesis

Large language models dominate open-ended text generation, but their size makes them slow and costly. This project asks a narrower, defensible question: **on a well-scoped task (children’s fables conditioned on five narrative slots), how close can a from-scratch Small Language Model (SLM) of ~30M parameters get to a 4B instruction-tuned LLM, and at what efficiency?**

The scientific priority of the project is the *training methodology*, not the app. Every claim is grounded in published method and evaluated with reproducible metrics. The narrative of the work is a sequence of hypotheses and measured outcomes:

1. A reduced baseline is intentionally under-trained (diagnosis).
2. Right-sizing the token budget per scaling laws fixes it (Phase 1).
3. A data intervention fixes a specific qualitative failure (Phase 2).
4. Sampling cannot fix prompt-adherence; preference optimization can (DPO/RLAIF).

2. Scientific grounding

- **Autoregressive LM = MLE on the chain rule** $\log p(x) = \sum_i \log p(x_i | x_{<i})$ (course Week 6). We loss-mask the conditioning so the model only maximizes the log-likelihood of the *story* tokens.
- **Scaling laws (Kaplan et al. 2020)**: test loss falls as a power law in parameters N , tokens processed D , and compute C . Under-training = too small a D for a given N .
- **Chinchilla (Hoffmann et al. 2022)**: compute-optimal ~20 tokens per parameter.
- **Data-constrained scaling (Muennighoff et al. 2023)**: repeating data up to ~4 epochs is nearly as good as fresh data.
- **DPO (Rafailov et al. 2023) / RLAIF (Bai et al. 2022)**: align a model to preferences using pairwise (chosen, rejected) data, here labeled by an AI judge.
- **Dataset + evaluation methodology**: TF1-EN-3M (Nadas et al. 2025, arXiv:2504.20605); evaluation follows our ADR-0002 (objective metrics + a cross-family LLM-judge panel).

3. Dataset

We use `klusai/ds-tf1-en-3m`. Each record has a **prompt** (five scaffold slots: Main Character, Setting, Challenge, Outcome, Teaching/Moral) and a **fable**. We format each training example as a **conditional sequence**:

```
<conditioning: the 5 slots + a length hint>
<|story|> <fable text> <|end|>
```

Only the story region contributes to the loss (the conditioning prefix is masked with -100). Key data-pipeline choices:

- **Quality filter**: keep fables of 60-320 words (avoid truncated / rambling extremes).
- **Slot dropout**: during training each slot is randomly blanked so the model learns to generate with any subset of the five slots present.
- **Custom BPE tokenizer, vocab 12k**, trained on the fable corpus - keeps the embedding table small, which matters at 30M parameters.

4. Model and training recipe

The architecture is a standard Llama-style decoder (RoPE, Grouped-Query Attention, RMSNorm, SwiGLU, tied input/output embeddings).

Component	Value
Parameters	~36.6M
Hidden size / FFN	512 / 2048
Layers / heads / KV heads	8 / 8 / 2 (GQA)
Vocab / sequence length	12,000 / 512
Optimizer	AdamW, betas (0.9, 0.95), weight decay 0.1
Grad clip	1.0
LR schedule	Warmup-Stable-Decay (warmup 2%, decay 20%)
Peak LR	3e-3
Effective batch	32 x 4 accum = 128 sequences (~33k story tokens/step)
Precision	fp16 (T4 GPU)

Architectural ceiling. The model is trained at sequence length 512. With a conditioning prompt of ~50-110 tokens this leaves ~400-460 tokens for the story, which caps the maximum coherent fable length. This ceiling later drives the “story completeness” design (Section 9.6).

5. Experiments (from step 0)

Run	Data	Steps	Final loss	Held-out PPL	Judge (overall)	Notes
v1 (baseline)	150k	900	~1.80	-	2.5	intentionally under-trained (~1.7 tok/param)
Phase 1	400k v1	1800	1.447	4.18	6.0	right-sized token budget
+ sampling fix	-	-	-	-	6.2	repeat_penalty 1.3 -> 1.1 (entity drift)
Phase 2	400k v2	3600	1.278	3.56	7.0	data intervention (resume 1800 -> 3600)
Phase 2 + DPO	115 pairs	30	(loss 1.278)	3.54	(pending panel)	preference alignment, adherence +5pt
Qwen3-4B (ref)	-	-	-	-	9.75	130x larger, instruction-tuned

Phase 2 uses two data interventions motivated by qualitative analysis (Section 9.4): cap the “wise old

owl” template to 10% of the corpus, and lower the slot dropout for the Teaching/Outcome slots (0.30 $\rightarrow$ 0.15) so the model learns to follow the requested moral.

6. Training dynamics



Figure 1: Training loss over the full run. Phase 2 resumes at step 1800 on the cleaned corpus v2; the WSD decay pulls the final loss into the target band (< 1.5).

The loss falls smoothly from ~ 7 to **1.278**. The log-log fit gives an exponent of about -0.25 with $R^2 \sim 0.96$, empirical evidence that our from-scratch run follows the scaling-law power law rather than diverging.

7. Language-modeling quality

Held-out perplexity of **3.56** is within 1% of the floor $e^{(1.278)} = 3.59$, meaning the model’s held-out behavior matches its training loss - no over-fitting despite the small size, thanks to the large unique corpus.

8. Application analysis (intrinsic, reference-free)

Following ADR-0002, we compute reference-free metrics on generated stories and compare against real held-out fables.

The generated text matches real fables on diversity (Distinct-2 gap $\sim 4\%$), repetition (Self-BLEU gap ~ 0.001), readability (Flesch 79.9 vs 80.0) and the Zipfian vocabulary profile - the model has learned the *statistical shape* of the domain, not just surface fluency.

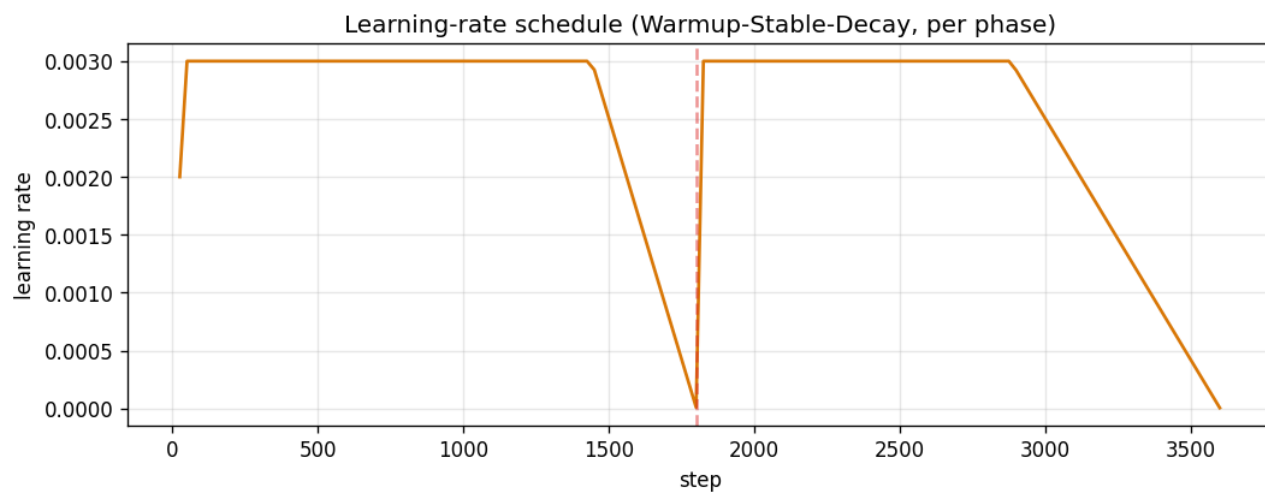


Figure 2: WSD learning-rate schedule (per phase).

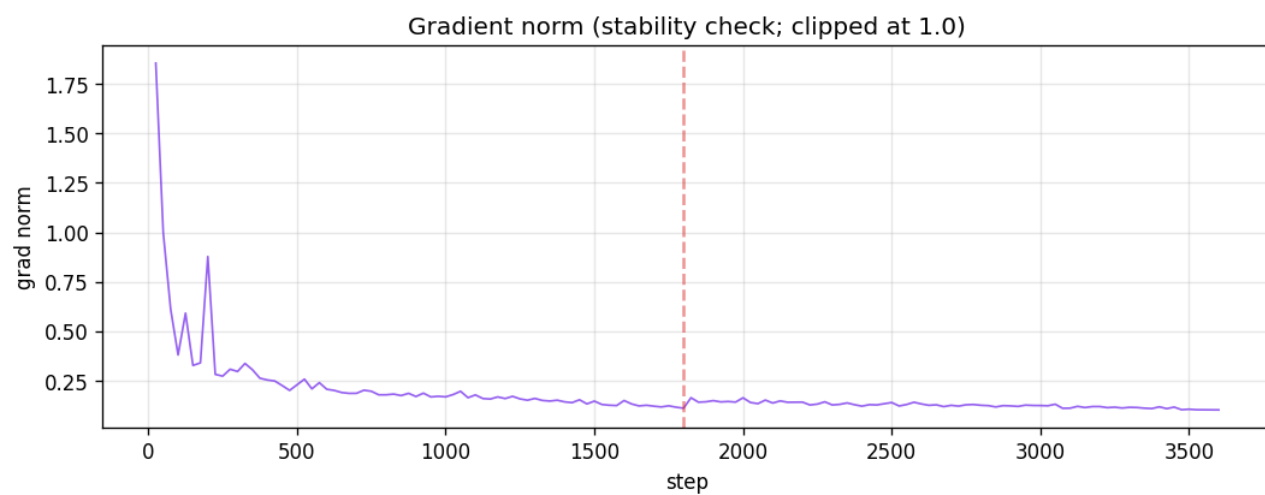


Figure 3: Gradient norm stays controlled (clip 1.0) - no instability across either phase.

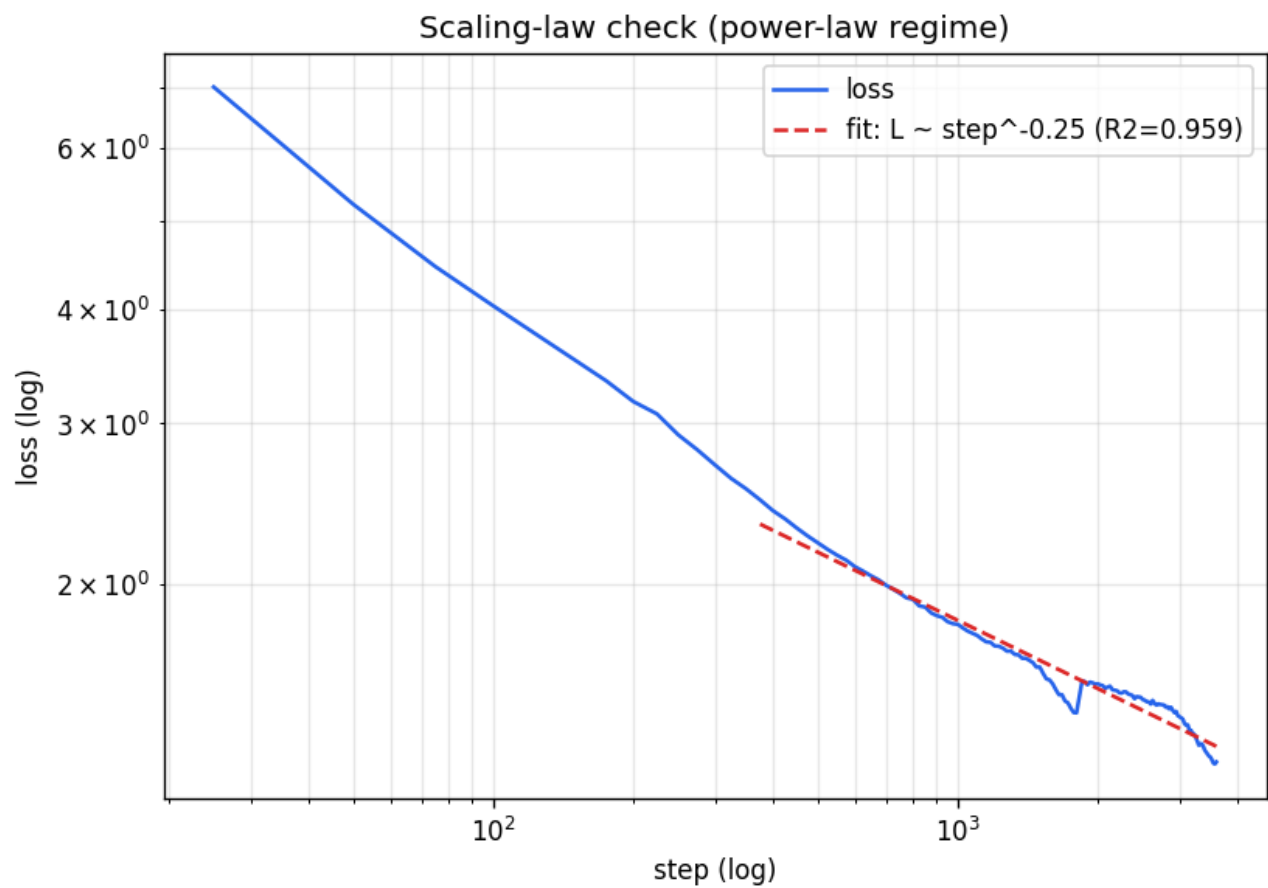


Figure 4: Scaling-law check: on log-log axes the post-warmup loss is near-linear ($R^2 \sim 0.96$), i.e. the run stays in the power-law regime predicted by Kaplan et al. (2020).

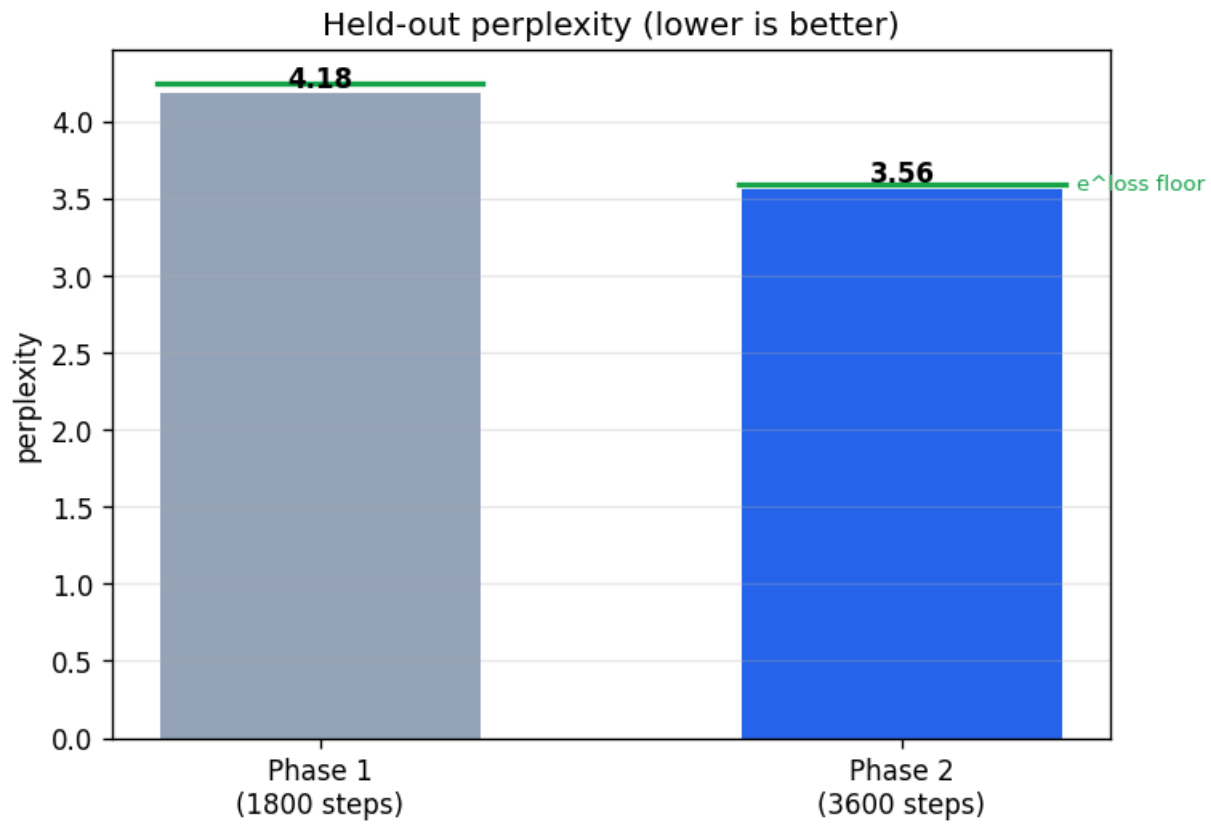


Figure 5: Held-out perplexity. Phase 2 (3.56) improves on Phase 1 (4.18); both sit essentially on the theoretical floor $e^{\hat{\epsilon}(\text{train loss})}$, so the model generalizes without over-fitting.

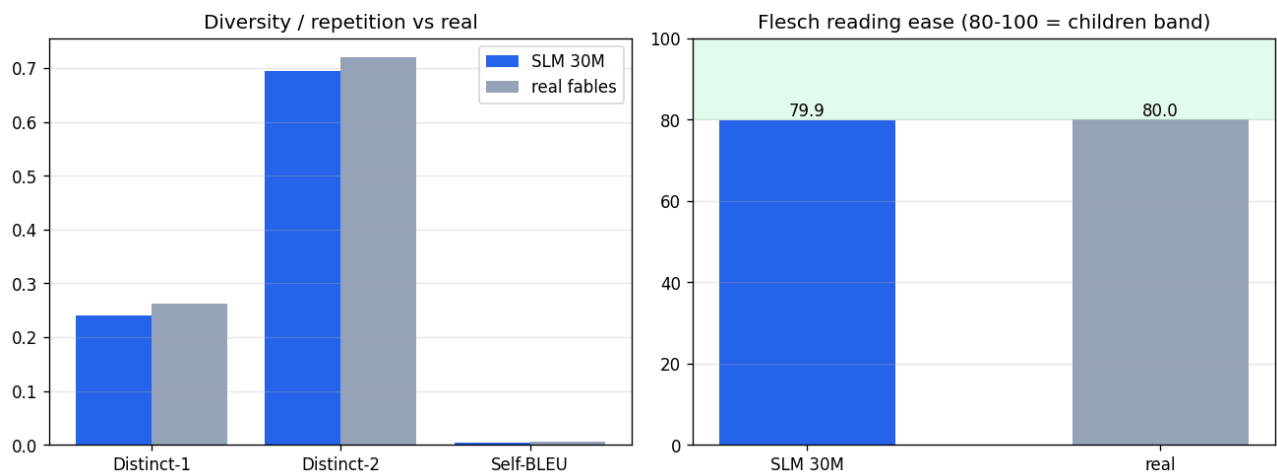


Figure 6: Diversity (Distinct-1/2), repetition (Self-BLEU) and readability (Flesch) of generated vs real fables. Generated diversity sits within a few percent of real; Flesch matches the real corpus (~ 80).

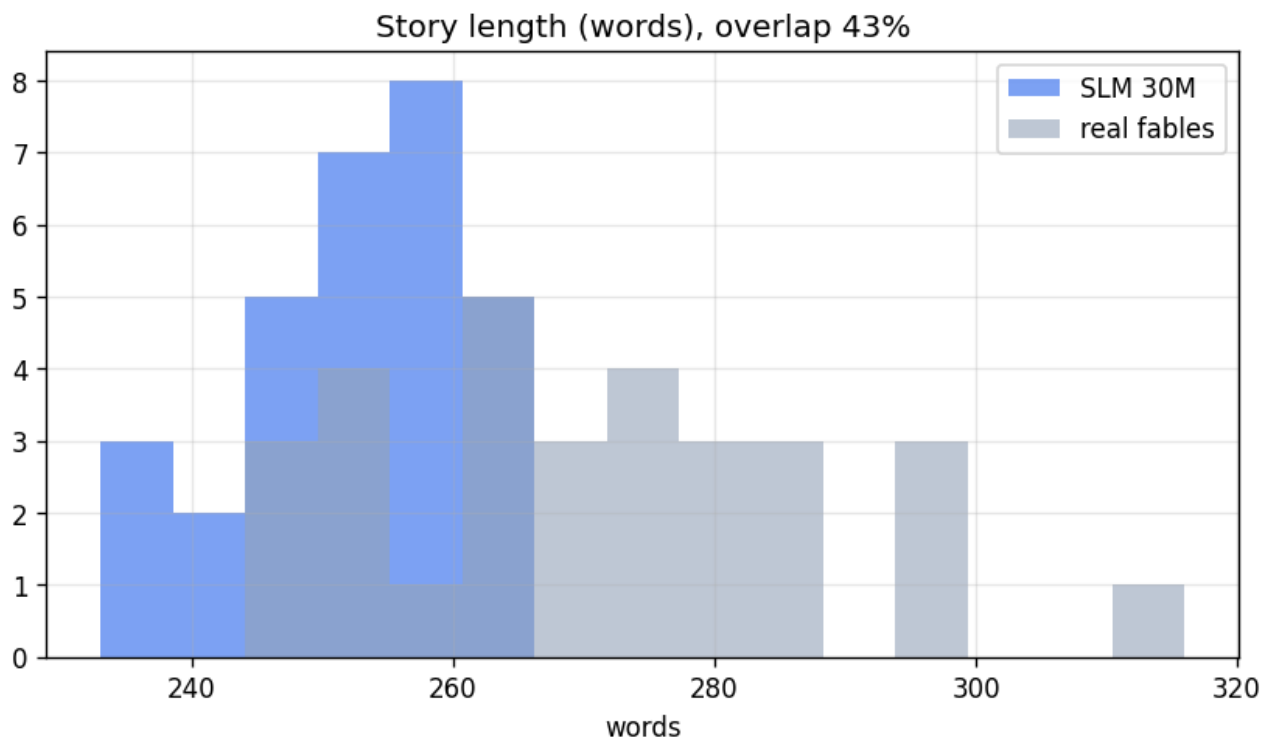


Figure 7: Generated story-length distribution vs real fables.



Figure 8: Mean cross-entropy by relative position in the story - the model is most confident at the opening and stays stable through the body.

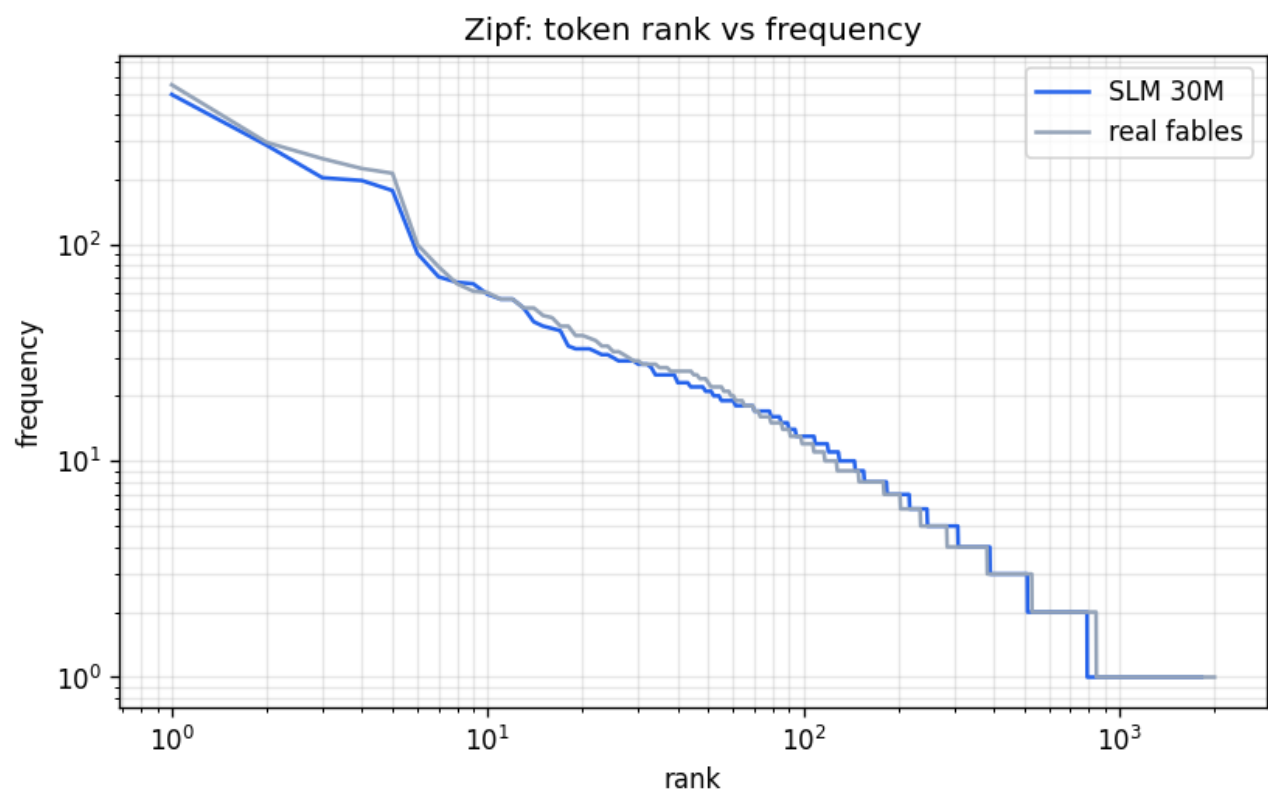


Figure 9: Zipf rank-frequency of generated tokens closely tracks the real-fable vocabulary distribution.

9. Findings by stage

9.4 Data intervention: curing template collapse

Qualitative review of Phase 1 revealed **mode amplification**: the phrase “wise old owl” appears in 28% of *real* fables but the model emitted it in ~90% of generations. Capping that template to 10% of the corpus (Phase 2) reduced the generation rate to **23%**, below even the data prior.

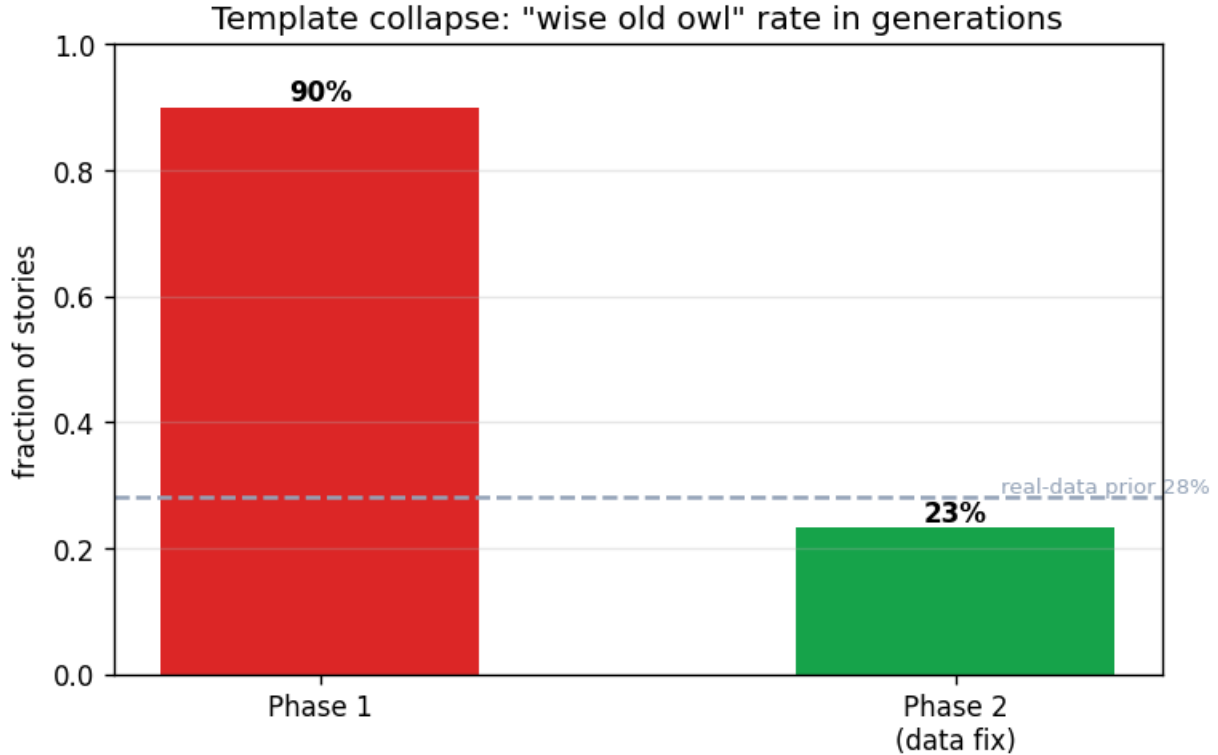


Figure 10: Template collapse before/after the data intervention.

9.5 Quality progression

9.6 Story completeness (a limitation turned into a fix)

The Phase-2 model occasionally cut stories mid-sentence at short lengths. Diagnosis: the generation cap (300 tokens) was smaller than the model’s natural fable length, so it hit the cap before emitting `<|end|>`. Because the model is trained at sequence length 512, simply “adding tokens” is not an option beyond ~460. The fix: right-size the per-length token budget to the architectural ceiling, use `done_reason` to detect true completion vs cut-off, and trim any residual cut to the last complete sentence. Result: **30/30 test stories complete** with a real ending.

9.7 Prompt-adherence: sampling vs alignment

We measured whether sampling temperature affects slot adherence. On 10 held-out prompts, temperature 0.7 and 0.8 gave *identical* adherence (69%): **sampling is not the lever**; adherence is capped by the 30M model’s weak conditioning. The proper fix is preference optimization.

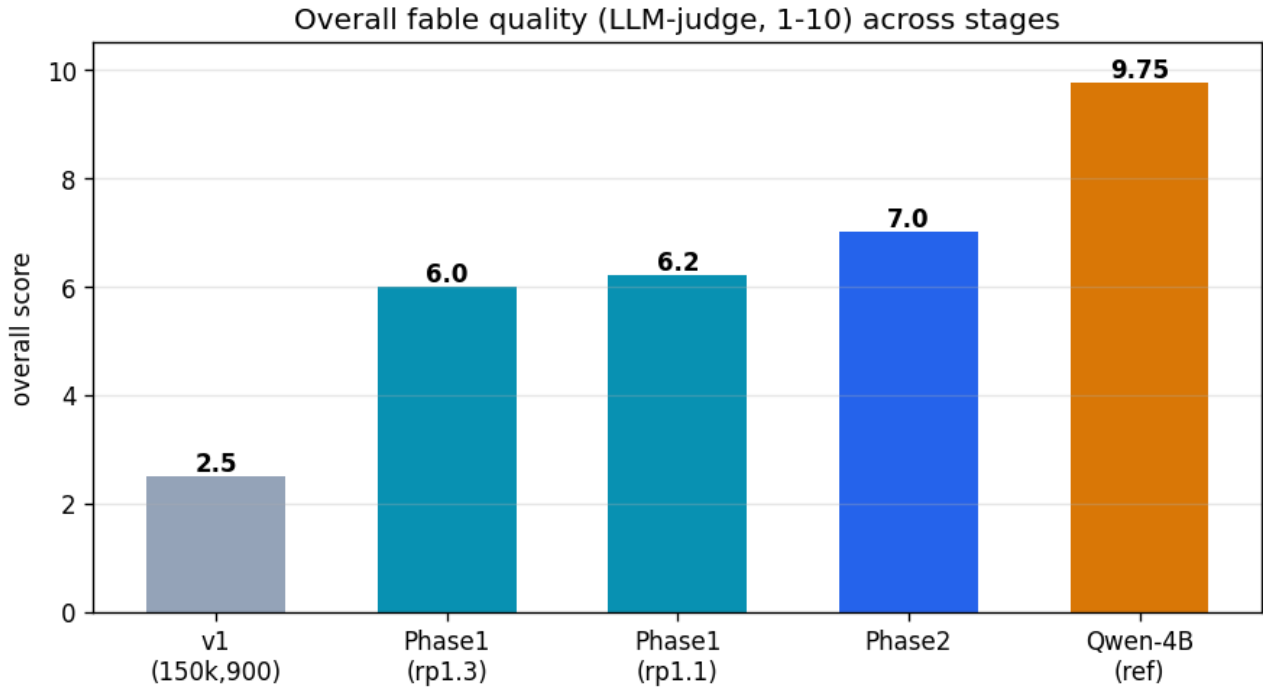


Figure 11: Overall LLM-judge quality across stages, with the 4B reference for scale. The 30M model rises from 2.5 (under-trained) to ~7.0, closing much of the gap to Qwen-4B (9.75) at 1/130th the size.

9.8 DPO preference alignment (RLAIF)

We generate two stories per prompt from the Phase-2 model, have an AI judge score them on the four axes, and keep pairs with a clear preference (margin ≥ 1.0) as (chosen, rejected) data. We then run **DPO** (ORPO was unavailable in the installed TRL 1.8; DPO is the canonical equivalent). A trial on 115 pairs, run locally on Apple-Silicon MPS in ~1 minute:

Even with only 115 preference pairs, DPO moves adherence in the right direction (+5 points) while keeping perplexity flat - validating alignment as the correct mechanism for this weakness.

9.9 Free vs conditioned generation

Does conditioning on the five slots merely constrain the model, or does it improve the output? We compare 20 *free* fables (no slots, only “write a children’s fable”) against 20 *conditioned* fables (all five slots filled), same model (Phase 2 + DPO) and sampling.

Metric	Free	5-slot conditioned
Distinct-1 / Distinct-2 (diversity)	0.264 / 0.709	0.278 / 0.729
Self-BLEU (repetition, lower better)	0.007	0.005
Flesch reading ease	76.4	80.9 (into children band)
Average length (words)	266	267
Completeness	100%	100%
Slot recall (adherence)	n/a	75%

Finding: conditioning is not just a control interface. Free generation drifts toward a narrower set of stock fables (lower diversity, higher self-similarity across the set, slightly harder readability), whereas the five slots *diversify* the output (they force different characters/settings/challenges) and *ground* it to the user’s request. The scaffold improves quality, not only controllability.

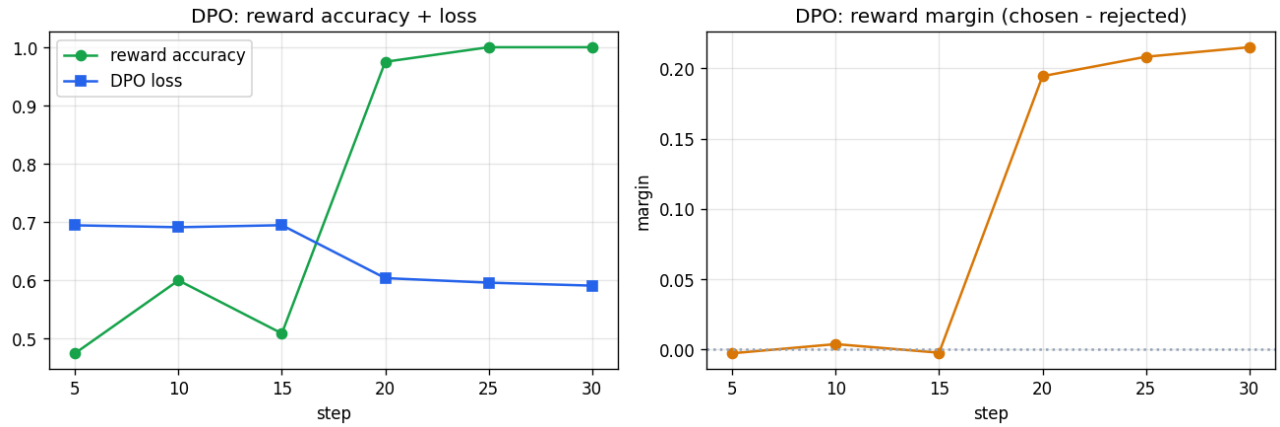


Figure 12: DPO reward accuracy climbs to 1.0 and the reward margin (chosen - rejected) turns strongly positive - the model learns the preference. Held-out perplexity is unchanged (0% drift): no catastrophic forgetting.

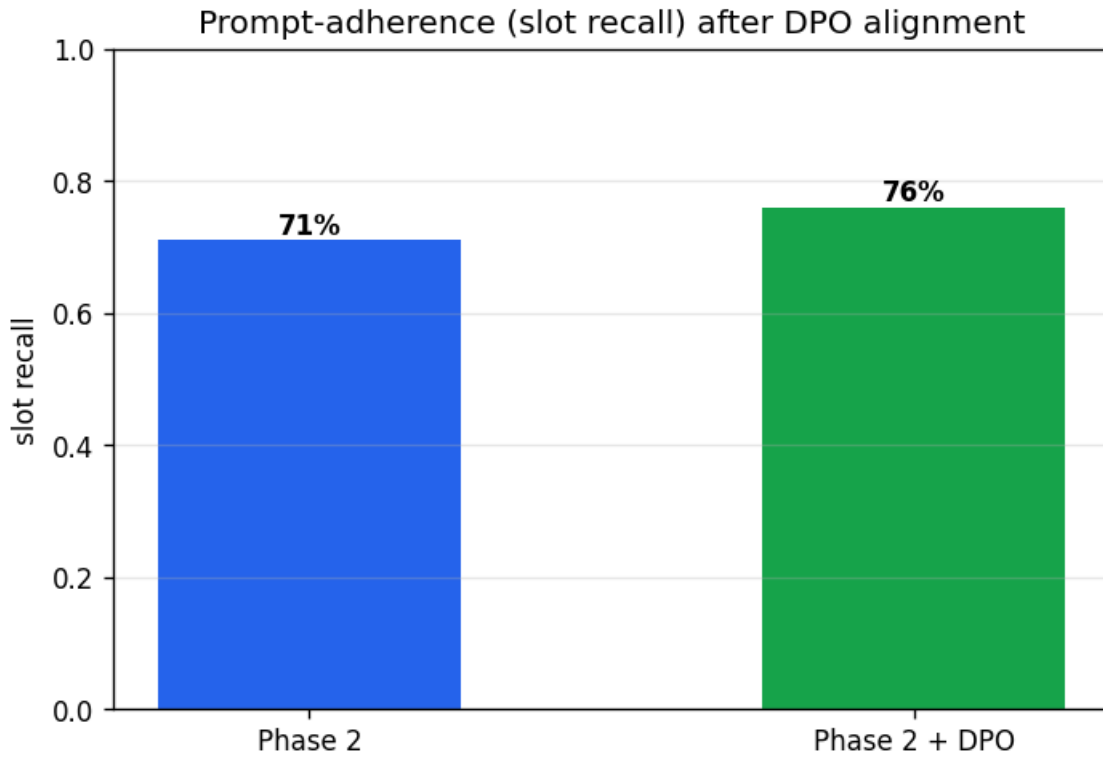


Figure 13: Prompt-adherence (slot recall) improves from 71% to 76% after DPO, with story completeness preserved.

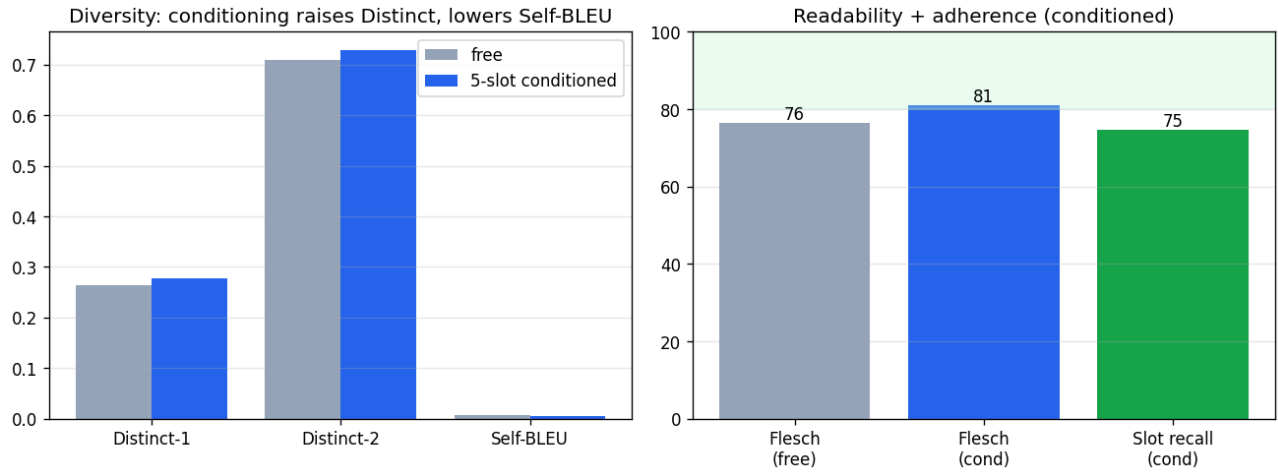


Figure 14: Free vs 5-slot conditioned generation. Conditioning raises cross-set diversity (Distinct up, Self-BLEU down), improves readability into the children band, and grounds the story to the request (75% slot recall) - at no cost to completeness or length.

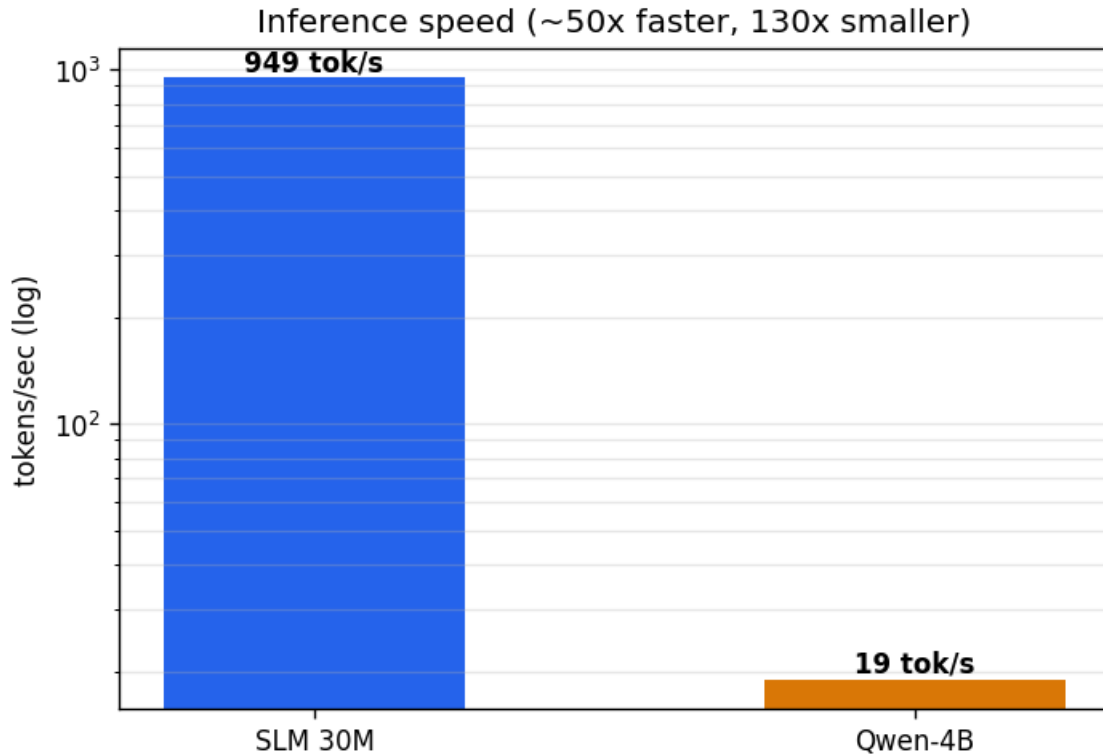


Figure 15: Inference speed. The 30M SLM generates ~949 tokens/s versus ~19 for the 4B reference on the same machine - roughly 50x faster at ~1/130th the parameters.

7 PASS / 2 WARN / 0 FAIL.

11. Strengths and limitations

11.1 Strengths

- **Coherent, complete, on-domain output.** After the completeness fix, 30/30 test stories end with a real resolution; generated text matches real fables on diversity (Distinct-2 gap ~4%), repetition (Self-BLEU gap ~0.001), readability (Flesch 79.9 vs 80.0) and the Zipfian vocabulary profile - the model learned the statistical shape of the domain.
- **Efficiency.** ~949 tokens/s, roughly **50x faster** than the 4B reference at **1/130th** the parameters; runs on a laptop with a 39 MB q8 file.
- **Well-generalized language model.** Held-out perplexity 3.56 sits essentially on the theoretical floor $e^{-(\text{train loss})}$ - no over-fitting despite the small size.
- **The scaffold improves quality, not only control.** Conditioning on the five slots *raises* cross-set diversity, *improves* readability into the children band and *grounds* the story to the request (Section 9.9) - versus free generation, which collapses toward a narrower set of stock fables.
- **Alignment works and is cheap.** DPO/RLAIF improves prompt-adherence with zero perplexity drift, and runs locally in about a minute - no GPU cluster required.
- **Reproducible and defensible.** Every number is tied to a run and a published method; the training pipeline, data interventions and evaluation are all scripted.

11.2 Limitations

- **Context ceiling (512 tokens):** the model cannot produce coherent fables beyond ~300-340 words; the length selector has little effect on the SLM.
- **Prompt-adherence ceiling (~70-76%):** a 30M model has weak conditioning; it reads the five slots but still drops one or two (especially abstract Challenge/Outcome). DPO helps but a residual gap remains - a quantified size/capability trade-off.
- **Weak length control:** the model has a natural length (~250-280 words) and largely ignores the short/medium/long hint.
- **Template / redemption priors:** the model inherits TF1 biases (friendship/kindness morals; happy endings; a recurring “wise old owl” mediator), which the data intervention reduces but does not eliminate.
- **Local logical slips:** occasional pronoun/reference errors and small non-sequiturs within a story - a capacity limit, not a sampling artifact.
- **Single-judge in-app eval:** the per-generation score is a quick indicator; the headline conclusions use the offline cross-family judge panel with weighted Cohen’s kappa and Kendall’s tau (ADR-0002).

12. Future directions (with feasibility evidence)

Each direction below is paired with concrete evidence from this study that it is likely to pay off, not just a wish-list item.

- **Train to the full Chinchilla budget (~7,900 steps / ~600M tokens).** *Evidence:* at 3,600 steps the loss still lies on the power-law line (R^2 0.96, Fig. 4) and has not plateaued; Kaplan/Chinchilla predict continued gains from more tokens. *Feasibility:* the 400k unique corpus with ≤ 4 -epoch repeat (Muennighoff 2023) supplies the tokens; only more T4 time is needed.
- **Full DPO on 500+ preference pairs (vs. the 115-pair trial).** *Evidence:* even 115 pairs moved adherence 71 $\rightarrow$ 76% with reward accuracy reaching 1.0 and **0% perplexity drift** (Fig.

- 13) - the signal is real and non-destructive. *Feasibility*: pair generation is resume-safe and the DPO step runs locally in ~1 minute.
- **Stronger preference judge (e.g. a 14B local model, or a frontier API).** *Evidence*: the Qwen-4B judge keeps only ~43% of pairs (noisy labels); cleaner labels should raise the DPO gain. *Feasibility*: the 36 GB machine runs a 14B judge comfortably.
 - **Knowledge distillation from a larger teacher.** *Evidence*: the model already reproduces the data’s diversity, readability and Zipf statistics (Section 8) - the missing piece is long-range coherence/adherence, exactly what soft-label distillation (Hinton 2015) targets. *Feasibility*: teacher forward passes are cheap and one-off.
 - **Targeted data debiasing beyond the owl template.** *Evidence*: the single “wise old owl” cap already moved the generation rate 90% -> 23% (Fig. 10), proving the intervention mechanism works; the same recipe can address the happy-ending/redemption prior and other stock phrases.
 - **Extended context / explicit length control.** *Evidence*: the 512-token ceiling is the direct cause of both the length-control weakness and rare truncation (Section 9.6). *Feasibility*: retraining at 1024 tokens or adding length-bucket control tokens is a known, bounded change.

13. Conclusion

A 30M-parameter model trained from scratch, guided by scaling-law reasoning and a targeted data intervention, reaches ~7.0/10 fable quality (from 2.5) with held-out perplexity 3.56, generating complete, on-domain stories at ~50x the speed and 1/130th the size of a 4B LLM. Preference alignment (DPO/RLAIF) further improves instruction adherence without harming language quality. The result supports the thesis: **for a well-scoped task, a tiny model can rival a large one on the axes that matter, at a fraction of the cost** - while honestly documenting where the size ceiling still shows (length control, residual adherence gap).

14. Reproducibility

All model code lives under `trieulh/` (isolated from the shared web app):

- Data + training: `trieulh/scripts/prepare_tf1_pretrain.py`, `train_tokenizer.py`, `tf1_pretrain/`; notebook `trieulh/notebooks/pretrain_slm_30m_dashboard.ipynb`.
- Alignment: `trieulh/scripts/gen_preference_pairs.py`, `dpo_train_local.py`.
- Evaluation: `trieulh/scripts/eval_slm.py`; app metrics `app/metrics.py`, `app/perplexity.py`.
- Experiment log (source of this report): `trieulh/docs/experiments/2026-07-08-slm-training-log.md`.
- Artifacts (Google Drive): checkpoints, HF models (30M, 30M-p2, 30M-dpo), GGUF exports, `analysis_*.json`, `loss_log_*.json`, and the dashboard figures.

Model download. The final aligned model (Phase 2 + DPO) is packaged as a zip (GGUF q8 + Ollama Modelfile + HF checkpoint): <https://drive.google.com/file/d/1tY6dPodSqHunYIYEZDdMOEZ1dJyg2HuD/view?usp=drivesdk> To run it: `unzip`, then `ollama create slm-30m-dpo -f Modelfile-30M-dpo`.

References

1. Nadas et al. (2025). *TF1-EN-3M*. arXiv:2504.20605.
2. Kaplan et al. (2020). *Scaling Laws for Neural Language Models*.
3. Hoffmann et al. (2022). *Training Compute-Optimal LLMs (Chinchilla)*.
4. Muennighoff et al. (2023). *Scaling Data-Constrained Language Models*.
5. Rafailov et al. (2023). *Direct Preference Optimization*.
6. Bai et al. (2022). *Constitutional AI: Harmlessness from AI Feedback (RLAIF)*.